

Generative AI for Non-Coders — Session 2 Workbook

Why Agents? The Case for Autonomous AI

By the end of these two hours, you will have a working **AI Pattern Advisor** agent — yours, customised to your job — that takes any business problem you describe and tells you which AI pattern fits: **Chatbot · RAG · Agent · Multi-Agent**.

No code. No prep. Just you, the EdYoda Agent Builder, and this workbook.

One rule for today: **you will not leave with notes. You will leave with a working agent.**



HOW TO USE THIS WORKBOOK IN THE ROOM

This workbook is a **build companion**, not a reading. You open it once when the facilitator calls “open your workbook to your track’s page” and then you work from it for the next hour.

What you’re actually building

You’re building an **agent as a pipeline of LLM nodes** inside the EdYoda Agent Builder. Each node has its own system prompt and runs in sequence — the output of Node 1 feeds Node 2, which feeds Node 3, and so on. By the end of the session your agent has **5 nodes** and runs the full sequence on one button-press.

That’s it. That’s the whole mental model. No code. No “tools” to toggle. Just a chain of small LLM brains, each with a clear job, wired in order.

The platform you’ll be in

After you log into build.edyoda.com, you’ll see an agent detail page with these sections:

SECTION	WHAT LIVES HERE
Overview	Agent name, the agent's role (<code>Description</code> field), welcome message, avatar emoji, file-upload toggle.
Configure → No-Code	The node pipeline. This is where most of your build happens.
Configure → Code	Ignore for today — that's for Python developers.
RAG Docs	Where you upload documents the agent retrieves from. This is how you give your agent persistent context.
Logs	Run history — useful after you publish.
Share Config (header button)	The public chat URL after you publish. Use this to test on your phone or share with a colleague.

The build, in five steps

STEP	WHAT YOU DO	TIME	GAP CLOSED (FROM SPRINT 1)
A	Create the agent. Add Node 1 : write a system prompt that uses web search .	~ 10 min	# 3 Tools
B	First test run. Send a starter problem to your one-node agent. Read the trace.	~6 min	— (proof-of-life run)
C	Add Node 2 to the pipeline. Its system prompt processes Node 1 's output.	~8 min	# 4 Autonomy
D	Upload a RAG Doc with your real context (company, audience, voice). Re-run.	~7 min	# 1 Memory
E	Add Nodes 3 , 4 , and 5 . Click Proceed (Generate Code) . Click Publish Agent . Run the published agent end-to-end.	~10 min	# 2 Multi-step + # 5 Planning

How the pages are laid out

- **Page 3** is the **Bakery Launch Agent** — the canonical 5 -node example the facilitator demos at the front of the room. Read it before you start your own build. Then copy whatever's useful into your track.
- **Pages 4 – 7** are the **four track pages**. Pick the one closest to your day job and work from that page only. Each page gives you copy-ready prompts for Nodes 1 – 5 .
- **Pages 8 – 11** are reference sheets (RAG-doc templates, troubleshooting, what to do when the pipeline misfires).

- **Page 1 2** is the **doctor track's sample patient cases** — use these in Step E if you'd rather not paste real patient context into a public tool.
- **Page 1 3** is your **week-after tracker** + the take-home reflection.

When something breaks during the build

The session has no separate help channel — it's just you, the facilitator, and the room. Before raising a hand, **try the workbook's troubleshooting first**:

- . The track page itself has a troubleshooting table for the 3 – 4 ways that specific pipeline tends to misbehave.
- . The reference pages (8 – 1 1) have generic troubleshooting for node-output, RAG-context, and publish-time failures.
- . If both of those don't get you unstuck in 6 0 seconds, **raise your hand in chat** (or unmute) and the facilitator will jump in. If your account is genuinely broken, the facilitator will share the **public chat URL** of their pre-built version of your agent — you use that to keep up with the room while you fix your own.

Pick your track now: 1 . Marketing/Sales → **page 4** 2 . Finance/Consulting → **page 5**
3 . Doctor/Healthcare → **page 6** 4 . Generic → **page 7**

BAKERY LAUNCH AGENT — the canonical 5-node example

This is the agent your facilitator demos at the front of the room. Same bakery from Session 1 — **Lumière**, in Mumbai, run by you and Sneha, launching a new croissant line next month. Last session a plain chatbot got two of six steps. This agent does all six.

Read this section before you start your own build. The track pages mirror this exact shape.

How to create it (you can publish this verbatim before class)

Step 1 — Create the agent. On the Agents list, click **+ New** and fill the modal:

- **Agent Name:** Bakery Launch Agent
- **Description:** *You are an end-to-end launch agent for Lumière, a Mumbai-based artisan bakery. Your job is to take a launch brief from the founder and return a publish-ready launch plan: live trend research, audience profile, a 7 -day social schedule with three posts in the brand voice, a 30 -day post-launch retro reminder, and an executive summary email. Brand voice is set by the partner, Sneha — funny, slightly sarcastic, never corporate. You work as a pipeline: each stage builds on the one before it.*
- **Welcome Message:** *Hi! I'm your launch agent. Tell me what you're launching — product, audience, launch date — and I'll come back with a full plan.*
- **Framework:** anthropic

Click **Create Agent** → **Confirm & Create**.

Step 2 — Configure → No-Code. Add five nodes, in this order:

NODE 1 — TREND SCOUT (CLOSES GAP #3 TOOLS)

- **Name:** Trend Scout
- **Model:** Claude Sonnet 4 .6
- **Max Tokens:** 1024
- **System Prompt:**

You are the Trend Scout for a Mumbai artisan bakery called Lumière.

Read the user's launch brief carefully. Then use web search to gather the freshest possible signal on:

- Current bakery and café trends in Mumbai (last 3 months)
- What's trending in the user's specific product category (e.g., croissants, sourdough, plant-based pastry) globally and in India
- Adjacent lifestyle trends that matter to the target audience (e.g., specialty coffee, brunch culture, food-photography aesthetics)

Output exactly this structure:

- HEADLINE TREND: [one sentence – the single most useful insight]
- LOCAL SIGNALS: [3 bullets, Mumbai-specific]
- CATEGORY SIGNALS: [3 bullets, product-category-specific]
- ADJACENT SIGNALS: [2 bullets – lifestyle/aesthetic adjacencies]
- SOURCES: [list 3-5 URLs you actually consulted]

Keep each bullet under 20 words. No filler. No "in conclusion." Just signal.

NODE 2 — AUDIENCE PROFILER (CLOSES GAP #1 MEMORY)

- **Name:** Audience Profiler
- **Model:** Claude Sonnet 4 .6
- **Max Tokens:** 1 0 2 4
- **System Prompt:**

You are the Audience Profiler for Lumière, a Mumbai artisan bakery.

The Trend Scout's output is your input. Combine it with what you know about Lumière from RAG Docs (brand voice, partner Sneha's tone, current customer base) and produce a tight audience profile for this specific launch.

Output exactly this structure:

- PRIMARY AUDIENCE: [one sentence – who buys this launch, where they live, what they earn, what they care about]
- WHY NOW: [2 lines – why this audience will care about THIS launch THIS month, grounded in the trends from Node 1]
- WHERE THEY ARE: [3 bullets – the channels/places where this audience actually pays attention, e.g., specific Instagram communities, neighbourhood cafes, food-blogger circles]
- WHAT NOT TO DO: [2 bullets – the moves that would alienate this audience. Be specific to Mumbai food culture.]

Voice: confident, no jargon. Write like a brand strategist briefing a founder over coffee, not like a market research report.

NODE 3 — CAMPAIGN PLANNER (CLOSES GAP #2 MULTI-STEP REASONING)

- **Name:** Campaign Planner
- **Model:** Claude Sonnet 4 .6

■ **Max Tokens:** 1 0 2 4

■ **System Prompt:**

You are the Campaign Planner for Lumière.

The Audience Profiler's output is your input. Build a 7-day social launch schedule with exactly three posts, written in Sneha's voice (funny, slightly sarcastic, never corporate, occasional Mumbai-isms welcome).

Output exactly this structure:

LAUNCH WEEK SCHEDULE

- Day -2 (Teaser): [channel] · [post time IST] · [hook in one line]
- Day 0 (Launch): [channel] · [post time IST] · [hook in one line]
- Day +3 (Social proof): [channel] · [post time IST] · [hook in one line]

POSTS (three, in Sneha's voice)

POST 1 – Teaser (Day -2)

[80-120 words. Must include a sensory detail about the croissant. Must end with a question or a tease, not a CTA.]

POST 2 – Launch (Day 0)

[80-120 words. Must reference one of the Mumbai-specific signals from Node 1. Soft CTA only – invite, don't sell.]

POST 3 – Social proof (Day +3)

[80-120 words. Built around a real-sounding customer reaction. End with a forward-looking line about what's next at Lumière.]

If the brief doesn't give you the product name, use [PRODUCT] as a placeholder. Don't invent a name.

NODE 4 — REMINDER DRAFTER (CLOSES GAP #5 PLANNING)

■ **Name:** Reminder Drafter

■ **Model:** Claude Sonnet 4 .6

■ **Max Tokens:** 1 0 2 4

■ **System Prompt:**

You are the Reminder Drafter for Lumière.

The Campaign Planner's output is your input. Draft a 30-day post-launch retrospective reminder that the founder will receive in a month.

Output exactly this structure:

REMINDER FOR: [launch date + 30 days, ISO format]

SUBJECT: [one line – punchy, specific to this launch]

BODY:

[8-12 lines. Tell the founder, in plain English, what data to pull and what questions to answer. Examples:

- Units sold of [PRODUCT] vs. forecast?
- Top-performing post of the three from launch week? By what metric?
- Did the Mumbai-specific angle from Day 0 outperform the generic teaser?
- Repeat-customer rate among Day 0 buyers?
- One thing Sneha would do differently next launch?

End with one line of encouragement, in Sneha's voice.]

This is a draft to be saved by the founder, not sent immediately.
Do not include "send now" or "schedule" instructions – that's the Summary Composer's job in the next node.

NODE 5 — SUMMARY COMPOSER (*CLOSES GAP #4 AUTONOMY*)

- **Name:** Summary Composer
- **Model:** Claude Sonnet 4 .6
- **Max Tokens:** 1 0 2 4
- **System Prompt:**

You are the Summary Composer for Lumière.

You receive the full output of Nodes 1-4 above. Your job is to compose a single executive brief that the founder can read in 90 seconds on Monday morning and immediately act on.

Output exactly this structure:

LUMIÈRE LAUNCH BRIEF – [PRODUCT, or [PRODUCT] if not specified]

Prepared by: Bakery Launch Agent · For: Lumière founders

THE TAKE (3 lines max)

[The headline insight from the whole run. What's the single thing the founder should know? What's the bet you're making?]

THE PLAN (the schedule from Node 3, condensed to 3 lines)

- Day -2: [hook + channel]
- Day 0: [hook + channel]
- Day +3: [hook + channel]

THE POSTS

[Embed the three full posts from Node 3, verbatim. Do not edit them.]

THE FOLLOW-THROUGH

[Embed the 30-day reminder from Node 4, verbatim.]

NEXT STEP FOR THE FOUNDER

[One sentence. The single thing the founder needs to decide or approve before the team can execute. Make it specific – "approve the Day 0 post copy" beats "review the plan".]

Sign off with: "– Lumière Launch Agent, run [today's date in ISO format]"

Step 3 — Click Proceed (Generate Code), then Publish Agent. The agent goes live and you get a public chat URL on the header (e.g., <https://build.edyoda.com/chat/<uuid>>). That URL is what you share with anyone you want to demo to — or with a stuck learner in the room.

Step 4 — Test it. Open the public chat URL and paste this brief:

Launching a new croissant line at Lumière next month – the "Monsoon Croissant", filled with masala chai cremeux and dusted with kala namak. Target: the Bandra-Khar lifestyle crowd. Sneha wants it on Insta-first, restaurant collabs second. Launch date: 14 July.

Watch the trace. Five nodes fire in order. You walk away with a full launch brief.

TRACK 1 — Marketing/Sales

Your agent will become: *AI Pattern Advisor for marketing/sales workflows.*

Build the same 5 -node shape as the bakery example, but pointed at your marketing/sales day job. The agent takes any marketing/sales problem you describe — “we get 2 0 0 leads a week and can’t qualify them,” “our campaign analytics live in 4 dashboards,” “we want a chatbot on the pricing page” — and returns a structured AI-pattern recommendation, customised to your context.

Step A — create the agent + Node 1 (Web Search Tool Agent)

Create the agent via the **+ New** modal:

- **Agent Name:** [YourName]'s AI Pattern Advisor – Marketing
- **Description:** *You are an AI Pattern Advisor for a marketing/sales team. When a user describes a business problem, you classify it as Chatbot, RAG, Agent, or Multi-Agent using a 4 -question funnel, and return a structured recommendation with maturity level and risk flags. You work as a pipeline.*
- **Welcome Message:** *Tell me a marketing or sales problem you’re trying to solve, and I’ll tell you what kind of AI pattern fits — and how to start.*

Then in **Configure → No-Code → Node 1** :

- **Name:** Web Search Tool Agent
- **System Prompt:**

You are the research stage of an AI Pattern Advisor for a marketing/sales team.

Read the user's problem statement carefully. Then use web search to find current best practices for solving problems of this shape. Look for:

- How leading B2B/B2C companies are using AI for this category of problem
- Specific tools, vendors, or open-source projects that solve this problem
- One or two short case studies (named company, named outcome)

Output exactly this structure:

- PROBLEM RESTATED: [one line – your understanding of what the user actually needs]
- CURRENT PRACTICE: [3 bullets – what good teams are doing in 2025]
- TOOLS NAMED: [3-5 tools/vendors by name, with one-line on each]
- CASE: [one short case study – company, what they did, what it produced]
- SOURCES: [3-5 URLs you actually consulted]

Keep each bullet under 25 words.

Step B — first run

Open the chat panel (the Test/Run area, or the published URL once you publish). Paste this starter problem:

```
We get 200 inbound leads per week. Each needs to be qualified, scored,
and routed to the right rep. What AI pattern should we use?
```

Expected output shape: the Trend Scout's structured output — a problem restatement, current practice, tools, a case, sources. If you instead get prose ("Great question! Lead qualification can be solved by..."), the structured-output instruction is being ignored — re-paste Node 1's system prompt.

Step C — add Node 2 (the Classifier)

Click **+ Add Node** in Configure → No-Code.

- **Name:** Pattern Classifier
- **System Prompt:**

```
You are the classifier stage of an AI Pattern Advisor for a marketing/sales team.
```

```
The Web Search Tool Agent's output is your input. Apply the 4-question
funnel in order, stop at the first YES:
```

1. Does the answer exist in a fixed document or database? YES → RAG.
2. Does the task need a single response with no follow-up action? YES → Chatbot.
3. Does the task require multiple steps, memory, or tool use? YES → Agent.
4. Does the task require multiple specialized capabilities working in parallel? YES → Multi-Agent.

```
Output exactly this structure:
```

- PATTERN: [Chatbot / RAG / Agent / Multi-Agent]
- MATURITY LEVEL: [L0 / L1 / L2 / L3]
- RATIONALE: [3 lines max. Plain English. No jargon.]
- RISK FLAG: [One line – what could go wrong if implemented poorly. Say "None material" only if truly low-risk.]

```
If the problem is ambiguous, ask ONE clarifying question before classifying.
```

Re-run the starter problem from Step B. Watch the trace — Node 1 fires, then Node 2 takes Node 1's research and produces the structured recommendation. **That's two nodes. That's your AI Pattern Advisor's spine.**

Step D — upload a RAG Doc with your context (this is your “memory”)

Go to the **RAG Docs** tab. Click upload and add a single text or PDF file containing your real context. The agent retrieves from this doc on every run.

Template (save as `my-context.txt` , edit before uploading):

```
Company: [X]
Audience: [Y]
Brand voice: [Z]
Top 3 priorities this quarter: [list 3]
Current marketing/sales stack: [list 3-5 tools]
```

Filled-in example (this is what “specific enough” looks like):

```
Company: Aperture – B2B SaaS in legal tech serving General Counsels
at mid-market US firms (200-2,000 employees).
Audience: technical buyers and skeptical legal ops leaders.
Brand voice: concise, data-led, slightly skeptical of vendor hype.
Top 3 priorities this quarter: 1) cut sales-cycle time by 25%,
2) double inbound MQL volume, 3) launch a self-serve trial.
Current marketing/sales stack: HubSpot, Salesforce, Apollo,
Outreach.io, ChiliPiper, Google Analytics.
```

Re-run the starter problem. The agent now grounds its recommendation in your actual stack and priorities — no more generic “use a CRM” suggestions.

Step E — add Nodes 3, 4, 5 + publish

Add three more nodes to extend the pipeline into a full advisory output:

- **Node 3 — Implementation Sketch:** outputs a 5 -step implementation plan for the recommended pattern, scoped to a 3 0 -day pilot.
- **Node 4 — Risk & Compliance:** flags the top 3 risks (data, customer trust, brand) and proposes mitigations.
- **Node 5 — Executive Brief:** composes everything into a one-page memo the user can paste into Slack or email.

Use the same prompt shape as the Bakery example’s Nodes 3 –5 — read the upstream node’s output, produce a tight structured output, hand off cleanly. You can copy the Summary Composer prompt from the Bakery section as a starting point for Node 5 .

Then: **Proceed (Generate Code) → Publish Agent**. Grab the public chat URL from the Share Config header button. Test it by pasting:

Our SDR team manually researches every inbound demo request before the discovery call. That's eating 30 minutes per rep per day. What pattern should we use?

Track-specific troubleshooting

WHAT'S HAPPENING	LIKELY CAUSE	THE FIX
Node 1 doesn't actually web-search; it just summarises from training data	Web search isn't being triggered. The system prompt needs an explicit "use web search" verb.	Open Node 1 , confirm the literal phrase "use web search" appears in the system prompt. If you wrote "research the web" or "find online", swap it for "use web search".
Node 2 classifies everything as "Agent"	Funnel order isn't being enforced — Node 2 is skipping to Q 3 because agents sound impressive.	Re-paste Node 2 's prompt. Confirm "Stop at the first YES. Do not skip questions." is still there.
Recommendations ignore my RAG-doc context	Either the doc didn't upload, or your nodes' system prompts don't reference the context.	Confirm the doc shows up under the RAG Docs tab. Then add a line to Node 2 's prompt: "Use the company context from RAG Docs to ground the rationale and risk flag."
Node 5 (Executive Brief) is too long	"One-page memo" wasn't constrained tightly enough.	Add to Node 5 's prompt: "Hard cap: 2 5 0 words total. Use bullets, not paragraphs. The reader has 9 0 seconds."

One-Line Reflection

What's the most surprising thing your published agent did during BUILD STEP E?

TRACK 2 — Finance/Consulting

Your agent will become: *AI Pattern Advisor for financial workflows.*

Same 5 -node shape, pointed at your finance/audit/compliance/consulting day job. The agent deliberately defaults to conservative reads on explainability and audit risk — when a workflow touches regulated data, the recommendation includes a Human-in-the-Loop checkpoint by default.

Step A — create the agent + Node 1

Via the + **New** modal:

- **Agent Name:** [YourName]'s AI Pattern Advisor – Finance
- **Description:** *You are an AI Pattern Advisor for a finance/consulting team. You default to conservative reads on explainability and audit risk. When a workflow touches regulated data, you always recommend Human-in-the-Loop. You work as a pipeline.*
- **Welcome Message:** *Tell me a finance, audit, or compliance workflow problem, and I'll tell you what kind of AI pattern fits — with explicit risk flags.*

Node 1 — Web Search Tool Agent system prompt:

You are the research stage of an AI Pattern Advisor for a finance/consulting team.

Read the user's problem statement. Then use web search to find current regulatory and industry guidance for the workflow described. Look for:

- Current regulator guidance (SEC, FINRA, OCC, FCA, RBI, SEBI – match jurisdiction)
- How major banks, asset managers, or consultancies are deploying AI here
- Audit-trail and explainability requirements specific to this workflow

Output exactly this structure:

- PROBLEM RESTATED: [one line]
- REGULATORY CONTEXT: [3 bullets – named regulators, named rules]
- CURRENT PRACTICE: [3 bullets – what regulated firms are actually doing]
- AUDIT-TRAIL CONSTRAINT: [one line – what must be preserved]
- SOURCES: [3-5 URLs]

Step B — first run

Our compliance team manually reviews 500 transactions per day for SOX.
What AI pattern fits – chatbot, RAG, agent, multi-agent?

Expected output shape: structured Node 1 output that names SOX, names the audit-trail requirement, and links to current SEC/PCAOB guidance.

Step C — add Node 2 (Classifier with regulated-data guardrail)

You are the classifier stage of an AI Pattern Advisor for a finance/consulting team. Be conservative on explainability and audit-trail risk.

Apply the 4-question funnel in order, stop at the first YES:

1. Does the answer exist in a fixed document or database? YES → RAG.
2. Single response, no follow-up action? YES → Chatbot.
3. Multiple steps, memory, or tool use? YES → Agent.
4. Multiple specialized capabilities in parallel? YES → Multi-Agent.

Output exactly this structure:

- PATTERN: [Chatbot / RAG / Agent / Multi-Agent]
- MATURITY LEVEL: [L0 / L1 / L2 / L3]
- RATIONALE: [3 lines max. Plain English. No finance jargon for jargon's sake.]
- RISK FLAG: [One line – call out explainability, audit, or regulatory risk explicitly if it applies.]

If the problem touches regulated data (PII, financial records, customer money), always flag it and recommend a Human-in-the-Loop checkpoint added to the recommended pattern (e.g., "Agent + HITL").

Step D — upload your context as a RAG Doc

Template (save as `my-context.txt`):

Sector: [e.g., private credit, retail banking, audit]
Portfolio focus: [X]
Key risk metrics: [Y]
Regulatory bodies you report to: [list]
Audit-trail requirements: [in one line]

Filled-in example:

Sector: private credit
Portfolio focus: mid-market direct lending (\$25M-\$150M tickets, mostly US sponsor-backed deals).
Key risk metrics: leverage multiple, interest coverage, covenant headroom, revenue concentration in top 3 customers.
Regulatory bodies you report to: SEC (as a registered investment advisor), state-level lending licenses.
Audit-trail requirements: every recommendation must be reproducible from documented inputs; LLM outputs must be reviewable by a human analyst within 24 hours of decision.

Step E — add Nodes 3, 4, 5 + publish

- **Node 3 — Implementation Sketch:** scoped to a 90 -day regulated-environment pilot, not 30 days.
- **Node 4 — Risk & Compliance:** ALWAYS includes named regulators and named audit-trail mitigations.
- **Node 5 — Executive Brief:** the audience is the CFO or Head of Compliance, not the GTM team. Voice is conservative.

Publish, grab the URL, and test with:

We want to use AI to summarise quarterly 10-K filings across our portfolio companies for our IC memo. What pattern fits?

Track-specific troubleshooting

WHAT'S HAPPENING	LIKELY CAUSE	THE FIX
RISK FLAG keeps saying "None material" on regulated workflows	The HITL trigger is being interpreted too softly.	Strengthen Node 2 : change <i>"If the problem touches regulated data..."</i> to <i>"If the problem touches any of: customer money, customer PII, regulated financial records, audit-traced workflows — always flag and always recommend HITL."</i>
Agent recommends full autonomy on a transaction-touching task	The HITL default isn't carrying through to Node 5 .	Add to Node 5 's prompt: <i>"On any workflow flagged by Node 2 as touching regulated data, the recommended pattern in the executive brief must be the upstream pattern + HITL, not the upstream pattern alone."</i>
RATIONALE is full of jargon (RWA, NPV, MD&A)	Plain-English instruction is being ignored.	Tighten Node 2 's output line: <i>"RATIONALE must be readable by a non-finance executive. Define any acronym on first use, or replace it with a plain phrase."</i>
Node 3 's implementation plan is too aggressive (e.g., "launch in production in 30 days")	Default cadence is too fast for regulated work.	Add to Node 3 : <i>"Default cadence: 90 -day regulated pilot, sandbox first, no production traffic until compliance sign-off. Adjust only if the user explicitly waives this."</i>

One-Line Reflection

What's the most surprising thing your published agent did during BUILD STEP E?

TRACK 3 — Doctor/Healthcare

Your agent will become: *AI Pattern Advisor for clinical workflows.*

Same 5 -node shape, pointed at clinical/ward/department workflows. This track has the strongest guardrails of any in the session — diagnosis, medication, and discharge workflows ALWAYS get Human-in-the-Loop, with no exceptions.

Step A — create the agent + Node 1

Via the + **New** modal:

- **Agent Name:** [YourName]'s AI Pattern Advisor – Clinical
- **Description:** *You are an AI Pattern Advisor for a clinical team. You default to Human-in-the-Loop for any workflow involving diagnosis, medication, or discharge. You are conservative on patient-data risk. You work as a pipeline.*
- **Welcome Message:** *Tell me a clinical workflow problem — handover, triage, documentation, scheduling — and I'll tell you what AI pattern fits, with explicit patient-safety flags.*

Node 1 — Web Search Tool Agent system prompt:

You are the research stage of an AI Pattern Advisor for a clinical team.

Read the user's problem statement. Then use web search to find current AI-assisted clinical workflow practices. Look for:

- Peer-reviewed studies or published case studies from academic medical centres
- Current regulatory guidance (FDA SaMD, NHS digital, EMA, CDSCO – match jurisdiction)
- Risks named in the literature: hallucination, bias, automation bias, alert fatigue

Output exactly this structure:

- PROBLEM RESTATED: [one line, in plain clinical English]
- CLINICAL PRECEDENT: [3 bullets – what academic medical centres are doing]
- REGULATORY POSTURE: [2 bullets – what regulators say about this category]
- NAMED RISKS: [3 bullets – specific risks the literature names]
- SOURCES: [3-5 URLs, prefer .gov, .edu, peer-reviewed journals]

Step B — first run

Our department spends 2 hours per day on patient handover notes between shifts. What AI pattern fits?

Expected output shape: Node 1 should return clinical precedent (e.g., Mayo, Mass General handover-summary pilots), regulatory posture, and named risks. If it returns generic SaaS-vendor marketing, the search bias is wrong — tighten Node 1 's prompt with *"prefer .gov, .edu, and peer-reviewed sources."*

Step C — add Node 2 (Classifier with clinical-safety guardrail)

You are the classifier stage of an AI Pattern Advisor for a clinical team. Be conservative on patient-data risk. Default to Human-in-the-Loop for any workflow involving diagnosis, medication, or discharge.

Apply the 4-question funnel in order, stop at the first YES:

1. Fixed document/database (e.g., guidelines)? YES → RAG.
2. Single response, no follow-up? YES → Chatbot.
3. Multiple steps, memory, tool use? YES → Agent.
4. Multiple specialised capabilities in parallel? YES → Multi-Agent.

Output exactly this structure:

- PATTERN: [Chatbot / RAG / Agent / Multi-Agent, with HITL appended where required]
- MATURITY LEVEL: [L0 / L1 / L2 / L3]
- RATIONALE: [3 lines max. Plain clinical English. Use clinical shorthand where standard (NEWS2, COPD, ED) but avoid management-consulting language.]
- RISK FLAG: [One line. Always name patient-safety, PHI, or regulatory risk explicitly. "None material" is only acceptable for purely administrative workflows.]

Never recommend full autonomy for any task touching diagnosis, medication, or discharge. Always append "+ HITL" to the pattern name for those.

Step D — upload your context as a RAG Doc

Template:

Department: [e.g., internal medicine, emergency, paediatrics]
Setting: [bed count, hospital type, jurisdiction]
Red-flag protocols: [list 3]
Patient-data constraints: [PHI rules in your jurisdiction]

Filled-in example:

Department: internal medicine, 28-bed step-down unit in a 600-bed academic tertiary hospital in Mumbai.

Setting: India, DPDP Act applies for patient data.

Red-flag protocols:

- 1) NEWS2 $\geq$ 5 triggers automatic rapid-response escalation
- 2) Any new oxygen requirement above baseline gets a senior review within 30 minutes
- 3) Any sepsis suspicion triggers the sepsis-six bundle within the hour

Patient-data constraints: No identified patient data leaves the hospital network. De-identified data may be used in research with IRB approval.

Default safety stance: always recommend HITL for diagnosis, medication, and discharge decisions.

Step E — add Nodes 3, 4, 5 + publish

- **Node 3 — Implementation Sketch:** scoped to a 6 -month clinical pilot with explicit IRB-style review checkpoints, NOT a 30 -day product launch.
- **Node 4 — Risk & Compliance:** every flag must name patient-safety, PHI, or regulatory risk explicitly. “None material” is rarely an acceptable answer here.
- **Node 5 — Executive Brief:** audience is the head of department or chief medical officer. The recommendation must be defensible in a clinical governance meeting.

Test with:

We want to triage incoming GP referrals (about 80 a day) to route urgent cases faster. What pattern fits?

Track-specific troubleshooting

WHAT'S HAPPENING	LIKELY CAUSE	THE FIX
Agent recommends full autonomy on a prescribing or discharge task	The HITL guardrail line was edited or got dropped on paste. Worst failure mode in this track.	Stop. Re-paste Node 2 's entire system prompt. Confirm the last two sentences about diagnosis/medication/discharge are intact. Re-run to verify HITL appears in the pattern name.
RISK FLAG keeps saying "None material" on patient-touching workflows	The "None material" exception is being interpreted liberally.	Tighten Node 2 : <i>"RISK FLAG must explicitly name a patient-safety, PHI, or regulatory consideration. 'None material' is only acceptable if the workflow does not touch patient identifiers, clinical decisions, or regulated reporting — and even then, justify it in the same line."</i>
RATIONALE reads for hospital executives, not clinicians	Audience is being inferred from search results, not your instruction.	Add to Node 2 : <i>"RATIONALE audience: another clinician on the ward. Use clinical shorthand where standard. Avoid management-consulting language."</i>
Node 5 's brief omits the patient-safety RISK FLAG	The brief is compressing for length and dropping safety-critical sections.	Add to Node 5 : <i>"On clinical workflows, the RISK FLAG section from Node 2 must appear verbatim in the executive brief. Do not compress or rewrite it."</i>

One-Line Reflection

What's the most surprising thing your published agent did during BUILD STEP E?

TRACK 4 — Generic

Your agent will become: *AI Pattern Advisor for general knowledge-work workflows.*

Same 5 -node shape, pointed at any office workflow that doesn't fit cleanly into marketing, finance, or healthcare — reporting, scheduling, status updates, vendor management, internal comms. This is the most flexible track and the best one if you're in a cross-functional role.

Step A — create the agent + Node 1

Via the + **New** modal:

- **Agent Name:** [YourName]'s AI Pattern Advisor – Generic
- **Description:** *You are an AI Pattern Advisor for any office function. You take any workflow problem and classify it as Chatbot, RAG, Agent, or Multi-Agent. You work as a pipeline.*
- **Welcome Message:** *Tell me a workflow problem from your day job and I'll tell you what AI pattern fits.*

Node 1 — Web Search Tool Agent system prompt:

You are the research stage of an AI Pattern Advisor for general office workflows.

Read the user's problem statement. Then use web search to find current best practices and tools for solving problems of this shape. Look for:

- How leading teams in similar functions are using AI for this workflow
- Specific tools, vendors, or open-source projects (be concrete)
- One short case study (named company, named outcome)

Output exactly this structure:

- PROBLEM RESTATED: [one line]
- CURRENT PRACTICE: [3 bullets]
- TOOLS NAMED: [3-5 tools/vendors by name, one line each]
- CASE: [one short case study]
- SOURCES: [3-5 URLs]

Step B — first run

Our team manually compiles weekly status updates from 5 different sources (Slack, Jira, Google Docs, email, a spreadsheet). What AI pattern fits?

Expected output shape: structured research output that names actual tools (Reclaim, Zapier, n8n, Notion AI, etc.). If the output is generic (“use an LLM to summarise”), strengthen Node 1’s “name specific tools and vendors” instruction.

Step C — add Node 2 (Classifier)

You are the classifier stage of an AI Pattern Advisor for general office workflows.

Apply the 4-question funnel in order, stop at the first YES:

1. Fixed document or database? YES → RAG.
2. Single response, no follow-up? YES → Chatbot.
3. Multiple steps, memory, tool use? YES → Agent.
4. Multiple specialised capabilities in parallel? YES → Multi-Agent.

Output exactly this structure:

- PATTERN: [Chatbot / RAG / Agent / Multi-Agent]
- MATURITY LEVEL: [L0 / L1 / L2 / L3]
- RATIONALE: [3 lines max. Plain English. No jargon.]
- RISK FLAG: [One line – what could go wrong. "None material" requires explicit justification in the same line.]

If the problem is ambiguous, ask ONE clarifying question before classifying.

Step D — upload your context as a RAG Doc

Template:

Function: [your role]
Company: [X]
Top 3 priorities this quarter: [list]
Current stack: [list 3-5 tools]

Filled-in example:

Function: operations at Plinth, a 120-person B2B SaaS in payroll.
Top 3 priorities this quarter:
1) cut quote-to-cash cycle time by 30%
2) automate the weekly executive status update
3) reduce manual data reconciliation between Salesforce and NetSuite
Current stack: Notion, Slack, Linear, Salesforce, NetSuite, Zapier.
Constraint: no engineering capacity for custom integrations beyond Zapier-level work.

Step E — add Nodes 3, 4, 5 + publish

- **Node 3 — Implementation Sketch:** 30-day pilot scoped to the constraint named in the RAG doc (e.g., Zapier-only, no engineering).
- **Node 4 — Risk & Compliance:** name at least one specific failure mode (e.g., “silent data staleness”, “tool quota exhaustion”, “permissions sprawl”).
- **Node 5 — Executive Brief:** audience is a team lead or department head.

Test with:

Our weekly executive status update takes 4 hours every Friday because we manually pull from 6 different systems. What pattern fits?

Track-specific troubleshooting

WHAT'S HAPPENING	LIKELY CAUSE	THE FIX
Recommendations are very generic ("use an AI agent to streamline your workflow")	Description field on the Overview tab is too broad — the agent has no domain anchor.	Edit the Overview tab Description: add one sentence naming your function: <i>"...for an operations team in a B2 B SaaS company of ~1 0 0 people."</i> You'll see specificity jump immediately.
Node 2 picks "RAG" for almost everything that involves any document	Q 1 of the funnel is being read too liberally — <i>"answer exists in a document"</i> is being interpreted as <i>"a document exists somewhere related to this problem."</i>	Tighten Node 2 's Q 1 : <i>"Is this purely an information-retrieval task where the answer is fully contained in an existing document, with no follow-up action needed? YES → RAG."</i>
RISK FLAG defaults to "None material" too often	Generic track has the loosest risk language.	Append to Node 2 's output block: <i>"RISK FLAG must name at least one specific failure mode. 'None material' requires explicit justification in the same line."</i>
Node 5 's executive brief is too long	"Executive brief" wasn't constrained tightly.	Add: <i>"Hard cap: 2 5 0 words. Bullets, not paragraphs. The reader has 9 0 seconds."</i>

One-Line Reflection

What's the most surprising thing your published agent did during BUILD STEP E?

PIPELINE PATTERNS — what each node should do well

NODE	JOB IN THE PIPELINE	WHAT MAKES IT WORK
Node 1 (Research)	Read the user’s problem, hit the web, return structured findings.	Explicit “ <i>use web search</i> ” instruction. Output structure that downstream nodes can parse. Source URLs included.
Node 2 (Classify / Decide)	Apply a framework to Node 1 ’s output. Return a structured recommendation.	A strict numbered framework. A “stop at first YES” rule. A required output shape with named fields.
Node 3 (Plan / Implement)	Turn the recommendation into a concrete action plan.	Scoped to a realistic time horizon. Names specific steps. Acknowledges the constraints in the RAG doc.
Node 4 (Risk / Compliance)	Surface what could go wrong and how to mitigate.	Names specific failure modes, not generic risks. References regulators or data constraints where relevant.
Node 5 (Compose / Ship)	Assemble Nodes 1 – 4 into a single artifact the user can act on.	Hard length cap. Specific audience. A clear “next step” the user can do today.

Why this shape works: each node has one job and one output structure. When the agent misbehaves, you know exactly which node to fix. When you want to add capability, you add a node — you don’t rewrite the existing ones.

RAG DOCS REFERENCE — how to write a good context doc

You upload one or more docs to the **RAG Docs** tab. The agent retrieves from these on every run. This is the closest thing to “persistent memory” in the platform.

What makes a RAG doc useful:

- **Specific over abstract.** “B2B SaaS in legal tech serving GCs at mid-market firms” beats “we’re a SaaS company”.
- **Names over labels.** Name your stack (HubSpot, Salesforce, Apollo) rather than describing it (“CRM, email tool, prospecting tool”).
- **Constraints up front.** “No engineering capacity beyond Zapier-level integrations” stops the agent from recommending custom builds.
- **Voice when relevant.** For marketing/comms tracks, paste 2 - 3 example sentences in your brand voice.

All four templates in one place — pick yours, fill it, save it as `.txt` or `.pdf`, upload it.

Marketing/Sales:

```
Company: [X]
Audience: [Y]
Brand voice: [Z]
Top 3 priorities this quarter: [list]
Current marketing/sales stack: [list 3-5 tools]
```

Finance/Consulting:

```
Sector: [e.g., private credit]
Portfolio focus: [X]
Key risk metrics: [Y]
Regulatory bodies you report to: [list]
Audit-trail requirements: [one line]
```

Doctor/Healthcare:

```
Department: [X]
Setting: [bed count + hospital type + jurisdiction]
Red-flag protocols: [list 3]
Patient-data constraints: [PHI rules in your jurisdiction]
```

Generic:

Function: [your role]
Company: [X]
Top 3 priorities this quarter: [list]
Current stack: [list 3-5 tools]

Sanity check after Step D:

- ☐ Doc shows up under the RAG Docs tab.
- ☐ Re-ran the starter problem from Step B.
- ☐ The output now references your real company / stack / constraints (not a generic example).

If the agent still gives generic output after upload: open Node 2 (or whichever downstream node should ground in context) and add the line *“Use the company context from RAG Docs to ground the rationale and risk flag.”*

🎯 MULTI-NODE PIPELINE — when to add Nodes 3, 4, 5

You add nodes when you want the pipeline to do *more*, not when one node isn't working. If Node 2 isn't classifying right, fix Node 2 — don't add Node 6 .

Add Node 3 (Plan/Implement) when: you want the agent to produce an actual action plan, not just a recommendation. This is the step that turns “you should use an Agent here” into “and here's how you'd ship a 30 -day pilot.”

Add Node 4 (Risk/Compliance) when: you want the agent to be honest about what could fail. This is especially important in finance and healthcare tracks where missing a risk is worse than over-flagging.

Add Node 5 (Compose/Ship) when: you want a single artifact the user can paste into Slack, email, or a doc and act on immediately. Without this node, your output is structured-but-fragmented; with it, you have a brief.

Generic troubleshooting — when your pipeline misfires

SYMPTOM	LIKELY CAUSE	THE FIX
A node skips its expected output structure	Output block is too long or too far from the start of the prompt	Move the output block to the bottom of the system prompt. Make sure it's clearly delimited (<code>Output exactly this structure: </code>).
The agent loops on Node 1 (re-searches multiple times)	Web-search instruction is too explicit, agent thinks it failed	Soften: “ <i>Use web search to find...</i> ” instead of “ <i>You MUST search for...</i> ”
Downstream node ignores upstream node's output	Downstream prompt doesn't reference its input	Add the literal line: “ <i>The [previous node name]'s output is your input. Read it carefully before you produce yours.</i> ”
Hallucinated sources, made-up companies in Node 1	No instruction to cite	Add: “ <i>Cite the URL of every source you reference. If no source is found for a claim, say so explicitly.</i> ”
Output is wrong format	System prompt got dropped on paste — only part of it landed	Re-paste the full system prompt from this workbook. The Output block is load-bearing.
Published agent's chat URL works but produces empty output	“Proceed (Generate Code)” wasn't clicked after the last node edit	Go back to Configure → No-Code, click Proceed , then unpublish and re-publish.

When in doubt: simpler prompt, fewer instructions, one fix at a time. Don't change three things and re-run — you won't know which one helped.

SAMPLE PATIENT CASE NOTES — Doctor Track Only

If you don't want to paste real patient data into a public tool during BUILD STEP E (good instinct), use one of these scrubbed sample cases. They are fictional, varied, and clinically plausible.

Case A — Post-op handover

Patient: 62yo M, POD#1 s/p elective laparoscopic cholecystectomy.
Vitals stable overnight, T 37.4, BP 128/78, HR 84, SpO2 97% on RA.
Pain 3/10 on PCA morphine. Tolerating sips. Mild nausea, ondansetron PRN.
JP drain output 40 mL serosanguinous. Ambulating with assistance.
Plan: advance diet, D/C PCA in AM, discharge home if tolerating PO and pain controlled.

Case B — ED admit

Patient: 78yo F, brought by family for acute confusion x 24h.
PMH: HTN, T2DM, mild cognitive impairment at baseline.
Vitals: T 38.6, BP 102/64, HR 110, RR 22, SpO2 94% on RA.
UA cloudy, +leuks, +nitrites. WBC 14.2. Lactate 2.1.
Likely urosepsis. Started IV ceftriaxone, IV fluids, sepsis bundle initiated.
Admit to medical floor with telemetry. Repeat labs in 6h.

Case C — COPD exacerbation

Patient: 67yo M with severe COPD (GOLD stage 3), 50 pack-year history.
Presents with 3-day worsening dyspnea, productive cough yellow sputum, low-grade fever.
On home O2 2L NC. Today SpO2 88% on 2L, increased to 4L NC, sat to 93%.
Started on prednisone 40mg PO daily x 5d, azithromycin 500mg day 1 then 250mg x 4d, duonebs q4h. CXR: hyperinflation, no focal infiltrate. No acute decompensation.
Plan: admit for 48h obs, wean O2 as tolerated, pulmonary consult.

Use any of these as the “problem” you ask your AI Pattern Advisor to classify — e.g., *“Should our handover-note workflow for cases like this be a Chatbot, RAG, Agent, or Multi-Agent system?”*

YOUR WEEK-AFTER TRACKER

This week, run your published AI Pattern Advisor **5 times** on real business problems from your work. Log each one here. We'll come back to this in Session 3 .

#	BUSINESS PROBLEM YOU CLASSIFIED	PATTERN THE AGENT PICKED	WAS IT RIGHT? (YES / PARTIAL / NO)	TIME SAVED VS. DOING IT YOURSELF	WHAT YOU'D CHANGE IN A NODE PROMPT
1					
2					
3					
4					
5					

End-of-week reflection (Friday, 5 minutes)

- . Which task was the biggest time saver? _____
- . Which task surprised you — either because the agent nailed it or because it was wrong in an interesting way? _____
- . Which task is **NOT** an agent task? (Should be a chatbot, a RAG system, or kept human?)

The third question is the most important one. The discipline of saying *“this is NOT an agent problem”* is what separates teams that ship from teams that get stuck.



TAKE-HOME (reply to today's class email by Friday)

Two sentences each. Your facilitator reads every reply.

- **ONE business problem at your job** that you'll classify with your AI Pattern Advisor this month. Tell me the problem in one sentence, and what pattern you think it'll come back as in the second.
- **ONE workflow you WON'T automate with an agent** — because it's a chatbot job, a RAG job, or because it's too sensitive for an agent today. One sentence on the workflow, one on why it stays manual (or stays a simpler pattern).

Why both questions matter: the first one builds the habit of using your agent. The second one builds the discipline of knowing when **not** to. Both are the work.



LINGERING QUESTIONS

Anything we didn't get to in the live Q&A: reply to today's class email. Answered by tomorrow morning. The biggest ones become opening callbacks in Session 3 .

WHAT'S NEXT — SESSION 3: PROMPT ENGINEERING

Today you built a 5 -node agent. Next time, we make every node's prompt sharp enough that the pipeline is reliable.

Session 3 — Prompt Engineering & Context Engineering for Agents is where the language you give each node gets precise enough to stop the small, annoying failures you'll spot over your 5 runs this week. You'll take the AI Pattern Advisor you built today and harden every node prompt — output schemas, refusal handling, edge-case behaviour. (Six more sessions ahead of this in the arc — RAG, agent architecture, the EdYoda Agent Builder deep-build, n 8 n workflows, and a business-cases capstone. We're in week 2 of the journey.)

Bring your tracker. Bring your favorite run, your weirdest run, and your worst run. We'll fix all three.

See you for Session 3 — Prompt Engineering. Go build.